Lab 4.1: Thread Synchronization 1

Total points: 100

Assignment 1 (10 points)

The following three processes execute concurrently on a shared processor. They can coordinate their execution using shared semaphores that support the standard **wait(S)** and **signal(S)** operations. Assume that execution may switch between any of the three processes at any point in time. The processes are defined as follows:

Process 1	**Process 2**	**Process 3**
Loop1: print("S")	**Loop2: print("E")**	**Loop3: print("T")**
print("W")	**goto Loop2**	**goto Loop3**
goto Loop1		

Requirements:

- Use the three processes given above for both below questions.

- You may introduce shared semaphores and insert **wait(S)** and **signal(S)** operations where necessary.

- Do not modify, remove, or reorder the original **print()** statements.

- Do not introduce additional processes or threads.

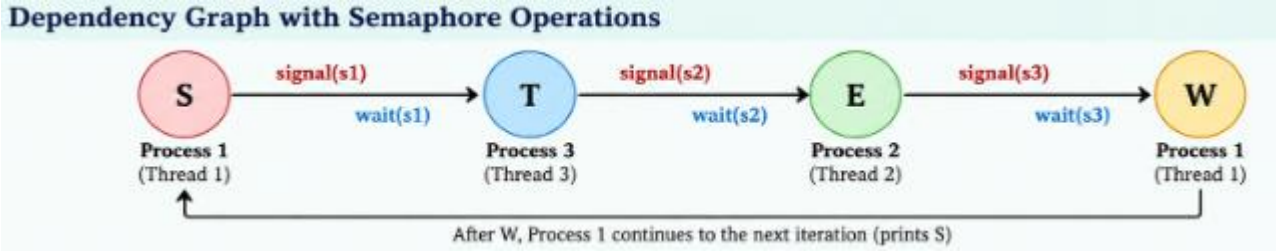
- In Java, use **java.util.concurrent.Semaphore** for synchronization.

a. Printing **STEW (5 points)**

Develop a Java program containing three threads that implement Processes 1, 2, and 3. The objective is to synchronize these threads using semaphores so that they repeatedly print the word **STEW**, regardless of their execution scheduling.

Expected output: STEWSTEWSTEWSTEWSTEW...

Hint:



b. Printing **SWEET (5 points)**

Develop a Java program containing three threads that implement Processes 1, 2, and 3. The objective is to synchronize these threads using semaphores so that they repeatedly print the word **SWEET**, regardless of their execution scheduling.

Expected output: SWEETSWEETSWEETSWEET...

For each of the following problems,

1. **Identify the correctness constraints of the problem**
2. **Specify the conditions that each method must wait for**
3. **Write down the shared state that you will use to check these conditions**
4. **Draw the UML class diagram**

Assignment 2 (10 points)

Develop a program which has a shared data and two threads. The shared data contains an integer variable which is set to 0 initially. The first thread will increase the variable a number of times (1000000 times) and the second thread will decrease the variable the same number of times (1000000 times). You should protect critical sections and thus prevent race conditions.

a. Using **synchronized** methods
b. Using **ReentrantLock** class
c. Using **Semaphore** class

The program creates, starts two threads and waits for the two threads to terminate and display the variable's value. Explain the result.

Assignment 3 (20 points)

Develop a **thread-safe** queue of integer (**TSQueue**). The thread-safe queue has two methods **addLast** and **removeFirst**. The **addLast** method inserts an integer value into the end of the queue and the **removeFirst** removes the first element of the queue. Test the class with several threads to insert and remove integer values into the queue.

Assignment 4 (20 points) – The Bounded-Buffer Problem

A bounded integer buffer is a fixed size FIFO queue and is implemented by an array of integer values. The queue has methods **add** and **remove**. The **add** method adds a new integer value to the end of the queue and the thread that call the **add** method must wait if the queue is full. The **remove** method removes the first element of the queue and the thread that call the **remove** method must wait if the queue is empty. Test the class with several threads to insert and remove integer values into the queue.

Assignment 5 (20 points) – The **MyBarrier** Class

A barrier allows a set of threads to all wait for each other to reach a common barrier point. The situation where the required number of threads have called **await()**, is called tripping the barrier.

- Constructor: **public MyBarrier (int parties)**
  + Creates a new **Barrier** that will trip when the given number of parties (threads) are waiting upon it.
  + parties: The number of parties (threads)
- **public void await()**:
  + Waits until all parties have invoked await on this barrier.

Write a **main** method to test the **MyBarrier** class.

Assignment 6 – Old Brigde (20 points)

An old bridge has only one lane and can only hold at most 3 cars at a time without risking collapse. Create a monitor with methods **arrive(int direction)** and **exit(int direction)** that controls traffic so that at any given time, there are at most 3 cars on the bridge, and all of them are going the same direction. A car calls **arrive** when it arrives at the bridge and wants to go in the specified direction (0 or 1); **arrive** should not return until the car is allowed to get on the bridge. A car calls **exit** when it gets off the bridge, potentially allowing other cars to get on.

a. Don't worry about starving cars trying to go in one direction; just make sure cars are always on the bridge when they can be. No cars should wait unnecessarily (the throughput will be maximized).

b. Instead of maximizing the throughput, the solution should guarantee that the starvation will not occur. That is, no cars should wait at the bridge forever.